AI-Use Declaration

Project: CpE211 Lab 5 — Stacks and Queues Using Arrays and Objects

Name	Kane Paul A. Mari
Course / Section	BSCpE-2
Instructor	Sir Jamie Eduardo Rosal
Date	10/24/2026

1. AI Tool and Model

GLM (model id *opencode/glm-5.3-flash-free*), accessed through the *opencode* command-line coding assistant. It was used strictly as a Socratic Java tutor: to propose pseudocode designs, quiz understanding through checkpoint questions, generate the final Java implementation only after the conceptual gates were passed, and draft documentation.

2. Prompts Used

- "Act as a Java tutor. Help me design pseudocode for Stacks and Queues Using Arrays and Objects. Do not write full code until I explain the algorithm back to you. Create ArrayStack and ArrayQueue classes; implement push/pop/peek and enqueue/dequeue/peek plus display and state checks; create a small Undo or customer-service simulation. Required methods: push(int x); pop(); peek(); isEmpty(); isFull(); enqueue(int x); dequeue(); display()."
- "For the manual trace and test evidence: trace PUSH 10, PUSH 20, POP for a stack and ENQUEUE A, ENQUEUE B, DEQUEUE for a queue; pop/dequeue from an empty structure; fill to capacity; push/enqueue then inspect; serve/remove all items in correct order."
- Checkpoint answers (the student's own explanations of push/pop index ordering, the *count* tiebreaker, the modulo wrap, and the undo order) submitted back to the tutor before any code generation was approved.
- "You can build now." — approval given after the design discussion and quiz gates were passed.
- "Make a readme.md for GitHub titled 'Stacks and Queues Using Arrays and Objects', plus two PDFs: one AI-use declaration, and one answering 'Why does a circular queue prevent wasted array space after dequeues?' Answer the question and verify it before building."

3. AI Suggestions Adopted

- Circular queue using *front*, *rear*, and *count* fields with the `(index + 1) % capacity` wrap.
- Guard-first pattern: *isEmpty()/isFull()* checked before every mutation; overflow rejected with a message, underflow/empty operations warn and return `-1`.
- Mapping "ENQUEUE A/B" to the integers 1/2 in traces, since the lab API is `int`-based.
- Menu-driven *Main* with a test-script option [9] that runs T1–T5 and prints PASS/FAIL lines as submission evidence.

- Single-file submission layout (`Lab5.java` hosting Main, ArrayStack, and ArrayQueue).

4. AI Suggestions Rejected or Modified

- **Rejected:** throwing exceptions (e.g., EmptyStackException) for empty structures — replaced by the lab-consistent `-1` sentinel plus a warning message.
- **Rejected:** shift-on-dequeue linear queue (kept only as a teaching comparison) in favor of the circular design.
- **Rejected:** one separate .java file per class — single file kept to match the lab submission format.
- **Modified:** trace wording and menu ordering simplified to match the required method list exactly.

5. Student Verification

- Explained both algorithms back before any code was written: push increments *top* then stores, pop reads then decrements; *count* resolves the front/rear ambiguity; `(4+1)%5=0` wraps rear to slot 0; undo removes 9 then 8.
- Hand-traced PUSH 10, PUSH 20 → POP = 20 and ENQUEUE 1, ENQUEUE 2 → DEQUEUE = 1 before running anything.
- Compiled with `javac` (JDK 25) and executed the program; the manual menu flow matched predictions (undo removed 20, FIFO served 100 before 200); menu [9] reported ALL TESTS PASSED (22/22 checks covering T1–T5, including overflow rejection and the rear wrap 4→0).
- Reviewed every generated file (`Lab5.java`, README, PDFs) before submission.

Kane Paul A. Mari
BSCpE-2 — 10/24/2026

I confirm that the declarations above accurately describe how AI was used in this work.